AI ASSISTED CODING
ASSIGNMENT-9.1

2303A52341
A.Niteesh
B-32

Problem 1:

Consider the following Python function:
def find_max(numbers):
return max(numbers)

Task:

• Write documentation for the function in all three formats:
(a) Docstring
(b) Inline comments
(c) Google-style documentation
• Critically compare the three approaches. Discuss the advantages, disadvantages, and suitable use cases of each style.
• Recommend which documentation style is most effective for a mathematical utilities library and justify your answer.

CODE:

```
def find_max(numbers):
    """
    Returns the maximum value from a list of numbers.

    Args:
        numbers (list): A list of numeric values.
    Returns:
        int or float: The largest number in the list.
    """
    return max(numbers)
def find_max1(numbers):
    # Use Python's built-in max() function
    # It scans through the list and returns the largest element
    return max(numbers)
def find_max2(numbers):
    """
    Returns the maximum value from a list of numbers.
    Args:
        numbers (list of int or float): The list of numbers to evaluate.
    Returns:
        int or float: The largest number in the list.
    Raises:
        ValueError: If the list is empty.
    """
    if not numbers:
        raise ValueError("The list is empty.")
    return max(numbers)
print("----Doc String----")
print(find_max.__doc__)
print("----Inline comments----")
print(find_max1.__doc__)
print("----Google Doc string----")
print(find_max2.__doc__)
```

OUTPUT:

```
----Doc String----

Returns the maximum value from a list of numbers.

Args:
    numbers (list): A list of numeric values.

Returns:
    int or float: The largest number in the list.

----Inline comments----
None
----Google Doc string----

Returns the maximum value from a list of numbers.

Args:
    numbers (list of int or float): The list of numbers to evaluate.

Returns:
    int or float: The largest number in the list.

Raises:
    ValueError: If the list is empty.
    numbers (list of int or float): The list of numbers to evaluate.

Returns:
    int or float: The largest number in the list.

Raises:
    ValueError: If the list is empty.
    ValueError: If the list is empty.
```

COMPARISON

Google-style documentation is most effective for a mathematical utilities library because it is professional, structured, and clear.

Aspect	Docstring	Inline Comments	Google-style
Readability	Good	Minimal	Excellent
IDE Support	Native	None	Native
Formality	Standard	Casual	Professional
Maintenance	Easy	Tedious	Structured
Type Hints	Optional	Unclear	Explicit
Examples	Rare	Never	Common

Format	Advantages	Disadvantages
Docstring	IDE tooltips, help() function, simple	Lacks structure, minimal detail

Inline	Code-adjacent clarity, detailed logic	Not accessible to tools, clutters code
Google	Professional standard, type specs, examples	Verbose, requires more effort

Problem 2: Consider the following Python function:
def login(user, password, credentials):
return credentials.get(user) == password
Task:
1. Write documentation in all three formats.
2. Critically compare the approaches.
3. Recommend which style would be most helpful for new developers onboarding a project, and justify your choice.

CODE:

```
def login(user, password, credentials):
    """
    Validates a user's login attempt.
    Args:
        user (str): The username to check.
        password (str): The password provided by the user.
        credentials (dict): A dictionary mapping usernames to passwords.
    Returns:
        bool: True if the password matches the stored credentials, False otherwise.
    """
    return credentials.get(user) == password
def login1(user, password, credentials):
    # Look up the stored password for the given user
    # Compare it with the provided password
    # Return True if they match, otherwise False
    return credentials.get(user) == password
def login2(user, password, credentials):
    """
    Validates a user's login attempt.
    Args:
        user (str): The username to check.
        password (str): The password provided by the user.
        credentials (dict): A dictionary mapping usernames to passwords.
    Returns:
        bool: True if the password matches the stored credentials, False otherwise.
    Raises:
        KeyError: If the username is not found in the credentials dictionary.
    """
    if user not in credentials:
        raise KeyError(f"Username '{user}' not found.")
    return credentials[user] == password
print("----Doc String----")
print(login.__doc__)
print("----Inline comments----")
print(login1.__doc__)
print("----Google Doc string----")
print(login2.__doc__)
```

OUTPUT:

```
ts/python/9.1AIAC.py
----Doc String----

Validates a user's login attempt.

Args:
    user (str): The username to check.
    password (str): The password provided by the user.
    credentials (dict): A dictionary mapping usernames to passwords.

Returns:
    bool: True if the password matches the stored credentials, False otherwise.

----Inline comments----
None
----Google Doc string----

Validates a user's login attempt.
Args:
    user (str): The username to check.
    password (str): The password provided by the user.
    credentials (dict): A dictionary mapping usernames to passwords.
Returns:
    bool: True if the password matches the stored credentials, False otherwise.
Raises:
    KeyError: If the username is not found in the credentials dictionary.
```

COMPARISON:

Point	Inline	Docstring	Google Style
Clear for beginners	No	Yes	Yes, very clear
Structured	No	Somewhat	Yes
Professional	No	Medium	Yes
For big projects	No	Okay	Yes
Best for onboarding	No	Good	Best

Google style clearly explains inputs, outputs, and possible errors
Therefore, for onboarding new developers in a real project,
It scales well in real-world production systems.
It supports automated documentation generation.
It reduces ambiguity in collaborative projects.
Google-style documentation is the most helpful approach.

Problem 3: Calculator (Automatic Documentation Generation)
Task: Design a Python module named calculator.py and demonstrate automatic documentation generation.
Instructions:
1. Create a Python module calculator.py that includes the following functions, each written with appropriate docstrings:
o add(a, b) – returns the sum of two numbers
o subtract(a, b) – returns the difference of two numbers

o multiply(a, b) – returns the product of two numbers
o divide(a, b) – returns the quotient of two numbers
2. Display the module documentation in the terminal using Python's documentation tools.
3. Generate and export the module documentation in HTML format using the pydoc utility, and open the generated HTML file in a web browser to verify the output.

CODE:

```python
"""
Calculator.py
A simple calculator module demonstrating automatic documentation generation.
"""

def add(a, b):
    """
    Returns the sum of two numbers.
    Args:
        a (int or float): First number.
        b (int or float): Second number.
    Returns:
        int or float: The sum of a and b.
    """
    return a + b
def subtract(a, b):
    """
    Returns the difference between two numbers.
    Args:
        a (int or float): First number.
        b (int or float): Second number.
    Returns:
        int or float: The result of a - b.
    """
    return a - b
def multiply(a, b):
    """
    Returns the product of two numbers.
    Args:
        a (int or float): First number.
        b (int or float): Second number.
    Returns:
        int or float: The product of a and b.
    """
    return a * b
def divide(a, b):
    """
    Returns the quotient of two numbers.
    Args:
        a (int or float): Numerator.
        b (int or float): Denominator.
    Returns:
        float: The result of a / b.
    Raises:
        ZeroDivisionError: If b is zero.
    """
    if b == 0:
        raise ZeroDivisionError("Denominator cannot be zero.")
    return a / b
```

OUTPUT:

TERMINAL

```
Help on module Calculator:

NAME
    Calculator

DESCRIPTION
    Calculator.py
    A simple calculator module demonstrating automatic documentation generation.

FUNCTIONS
    add(a, b)
        Returns the sum of two numbers.

        Args:
            a (int or float): First number.
            b (int or float): Second number.

        Returns:
            int or float: The sum of a and b.

    divide(a, b)
        Returns the quotient of two numbers.

        Args:
            a (int or float): Numerator.
            b (int or float): Denominator.

        Returns:
            float: The result of a / b.

        Raises:
            ZeroDivisionError: If b is zero.

    multiply(a, b)
        Returns the product of two numbers.

        Args:
            a (int or float): First number.
            b (int or float): Second number.
```

```
        Returns:
            int or float: The product of a and b.

    subtract(a, b)
        Returns the difference between two numbers.

        Args:
            a (int or float): First number.
            b (int or float): Second number.

        Returns:
            int or float: The result of a - b.
```

HTML

index
Calculator c:\users\ankam\onedrive\documents\python\calculator.py

Calculator.py
A simple calculator module demonstrating automatic documentation generation.

Functions

add(a, b)
Returns the sum of two numbers.

Args:
a (int or float): First number.
b (int or float): Second number.

Returns:
int or float: The sum of a and b.

divide(a, b)
Returns the quotient of two numbers.

Args:
a (int or float): Numerator.
b (int or float): Denominator.

Returns:
float: The result of a / b.

Raises:
ZeroDivisionError: If b is zero.

multiply(a, b)
Returns the product of two numbers.

Args:
a (int or float): First number.
b (int or float): Second number.

Returns:
int or float: The product of a and b.

subtract(a, b)
Returns the difference between two numbers.

Args:
a (int or float): First number.
b (int or float): Second number.

Returns:
int or float: The result of a - b.

Problem 4: Conversion Utilities Module
Task:
1. Write a module named conversion.py with functions:
o decimal_to_binary(n)
o binary_to_decimal(b)
o decimal_to_hexadecimal(n)
2. Use Copilot for auto-generating docstrings.
3. Generate documentation in the terminal.
4. Export the documentation in HTML format and open it in a browser.
CODE:

```
"""
conversion.py
A utility module for number system conversions.
"""
def decimal_to_binary(n):
    """
    Convert a decimal integer to its binary representation.
    Args:
        n (int): Decimal number to convert.
    Returns:
        str: Binary string representation of the number.
```

```python
    """
    return bin(n).replace("0b", "")
def binary_to_decimal(b):
    """
    Convert a binary string to its decimal integer equivalent.
    Args:
        b (str): Binary string (e.g., '1010').
    Returns:
        int: Decimal integer value of the binary string.
    """
    return int(b, 2)
def decimal_to_hexadecimal(n):
    """
    Convert a decimal integer to its hexadecimal representation.
    Args:
        n (int): Decimal number to convert.
    Returns:
        str: Hexadecimal string representation of the number.
    """
    return hex(n).replace("0x", "").upper()
```

OUTPUT: Terminal

```
Help on module conversion:

NAME
    conversion

DESCRIPTION
    conversion.py
    A utility module for number system conversions.

FUNCTIONS
    binary_to_decimal(b)
        Convert a binary string to its decimal integer equivalent.

        Args:
            b (str): Binary string (e.g., '1010').

        Returns:
            int: Decimal integer value of the binary string.

    decimal_to_binary(n)
        Convert a decimal integer to its binary representation.

        Args:
            n (int): Decimal number to convert.

        Returns:
            str: Binary string representation of the number.

    decimal_to_hexadecimal(n)
        Convert a decimal integer to its hexadecimal representation.

        Args:
            n (int): Decimal number to convert.

        Returns:
            str: Hexadecimal string representation of the number.
```

HTML

index
conversion c:\users\ankam\onedrive\documents\python\conversion.py

conversion.py
A utility module for number system conversions.

Functions

binary_to_decimal(b)
Convert a binary string to its decimal integer equivalent.

Args:
b (str): Binary string (e.g., '1010').

Returns:
int: Decimal integer value of the binary string.

decimal_to_binary(n)
Convert a decimal integer to its binary representation.

Args:
n (int): Decimal number to convert.

Returns:
str: Binary string representation of the number.

decimal_to_hexadecimal(n)
Convert a decimal integer to its hexadecimal representation.

Args:
n (int): Decimal number to convert.

Returns:
str: Hexadecimal string representation of the number.

Problem 5 – Course Management Module
Task:
1. Create a module course.py with functions:
o add_course(course_id, name, credits)
o remove_course(course_id)
o get_course(course_id)
2. Add docstrings with Copilot.
3. Generate documentation in the terminal.
4. Export the documentation in HTML format and open it in a browser.
CODE:

course.py

Assignment 9.1 > course.py > remove_course

```python
# Create 3 functions add_course(course_id, name, credits), remove_course(course_id),
# get_course(course_id), with proper docstrings.
def add_course(course_id, name, credits):
    """
    Adds a course to the course list.

    Parameters:
    course_id (str): The unique identifier for the course.
    name (str): The name of the course.
    credits (int): The number of credits for the course.

    Returns:
    dict: A dictionary representing the added course.
    """
    course = {
        'course_id': course_id,
        'name': name,
        'credits': credits
    }
    return course

```

course.py

Assignment 9.1 > course.py > remove_course

```python
# Create 3 functions add_course(course_id, name, credits), remove_course(course_id),
# get_course(course_id), with proper docstrings.
def add_course(course_id, name, credits):
    """
    Adds a course to the course list.

    Parameters:
    course_id (str): The unique identifier for the course.
    name (str): The name of the course.
    credits (int): The number of credits for the course.

    Returns:
    dict: A dictionary representing the added course.
    """
    course = {
        'course_id': course_id,
        'name': name,
        'credits': credits
    }
    return course
```

```
PS Z:\AIAC\Assignment 9.1> python -m pydoc -w .\course.py

Adds a course to the course list.

Parameters:
course_id (str): The unique identifier for the course.
name (str): The name of the course.
credits (int): The number of credits for the course.

Returns:
dict: A dictionary representing the added course.


Removes a course from the course list.

Parameters:
course_id (str): The unique identifier for the course to be removed.
course_list (list): The list of courses from which to remove the course.

Returns:
bool: True if the course was successfully removed, False otherwise.


Retrieves a course from the course list.

Parameters:
course_id (str): The unique identifier for the course to be retrieved.
course_list (list): The list of courses from which to retrieve the course.

Returns:
dict: A dictionary representing the retrieved course, or None if the course is not found.

wrote course.html
```

Pydoc: module course

localhost:8080/course.html

course

```
# Create 3 functions add_course(course_id, name, credits), remove_course(course_id),
# get_course(course_id), with proper docstrings.
```

Functions

add_course(course_id, name, credits)

```
Adds a course to the course list.

Parameters:
course_id (str): The unique identifier for the course.
name (str): The name of the course.
credits (int): The number of credits for the course.

Returns:
dict: A dictionary representing the added course.
```

get_course(course_id, course_list)

```
Retrieves a course from the course list.

Parameters:
course_id (str): The unique identifier for the course to be retrieved.
course_list (list): The list of courses from which to retrieve the course.

Returns:
dict: A dictionary representing the retrieved course, or None if the course is not found.
```

remove_course(course_id, course_list)

```
Removes a course from the course list.

Parameters:
course_id (str): The unique identifier for the course to be removed.
course_list (list): The list of courses from which to remove the course.

Returns:
bool: True if the course was successfully removed, False otherwise.
```